

Guess That Movie By Rashid Zaheer, Freya Patel, and Andrew Rebits

Introduction

This project is a **movie-guessing and recommendation game** powered by machine learning. Users enter a one-sentence movie plot, and the algorithm attempts to guess the movie title. If the guess is incorrect, the user answers up to four additional questions, giving the model five chances in total to identify the movie. If the algorithm still fails, it recommends movies similar to the one the user had in mind.

The project's primary goal is **entertainment**—to engage users through an interactive game that combines natural language processing and recommendation systems. Even when the model guesses incorrectly, users benefit from discovering new movies that match their interests.

For supervised learning, the project uses a **LinearSVC model**, which unexpectedly outperformed more complex models in predicting movie titles. For recommendations, it employs a combination of **K-Means clustering** and **cosine similarity**, effectively grouping and ranking similar movies.

Unlike typical data science projects focused on finance or forecasting, this one demonstrates how **machine learning can be used for fun and creative applications**—bringing together data science and movie enjoyment.

Related work

Our project shares similarities with existing guessing and recommendation systems, but differs in structure and approach.

- **Akinator:** Like Akinator, which guesses a user's chosen object or character through unlimited yes/no questions, our project is also a guessing game. However, it is limited to five fixed questions and uses natural language processing (NLP) to interpret user inputs instead of dynamic questioning.
- **Filmillion:** Similar to Filmillion, which guesses movies through yes/no questions and learns from user feedback, our model also aims to identify movies. However, it does not learn new movies and instead transitions to movie recommendations after five attempts, distinguishing it from Filmillion's continuous guessing.
- **Easy-Peasy AI Recommender:** While Easy-Peasy simply recommends movies based on a favorite title and genre, our project begins as an interactive guessing game and applies NLP and unsupervised learning for more personalized and context-aware recommendations.

Data Sources

- **Hugging Face Dataset:** Provided Wikipedia movie plots/summaries, which were readily usable as pandas dataframes. The dataset includes movies from 1901 to 2017.
 - Movie plot summary:
<https://huggingface.co/datasets/vishnupriyavr/wiki-movie-plots-with-summaries>
- **IMDb Non-Commercial Datasets:** Offered extensive movie metadata, including titles, plots, quotes, directors, writers, ratings, and release years. These datasets required extraction, formatting, and loading into dataframes.
 - <https://datasets.imdbws.com/>

Key Variables Used: Movie Title, Release year, plot, genre, actors, directors

Dataset Sizes and Filtering:

- Wikipedia summaries: ~34.9k rows

- IMDb data: ~14 million rows

Filtering Criteria: Focused on American movies released after 1969 to ensure completeness of key variables. Final usable dataset: 8,691 movies suitable for supervised and unsupervised model training.

Feature Engineering:

Remove exact duplicates: Drop duplicate rows that share both **Title** and **ReleaseYear** to ensure one canonical record per movie.

Missing value treatment: Missing values for genre, actors, and directors were treated using a combination of IMDb datasets and the movie's plot dataset. The following are the steps

1. **Load IMDb reference tables**

Load the following IMDb TSV files :

- title.basics.tsv → genres (and basic title info)
- title.principals.tsv and name.basics.tsv → actors (principal cast → map to actor names)
- title.crew.tsv and name.basics.tsv → directors (crew → map to director names)

2. **Merge IMDb data into the movie's plot dataset**

- Left-join IMDb title.basics.tsv with the movie's plot using Title and Released Year keys
- Appended genres, actors, and directors into the movie-plot dataset so that plot rows gain IMDb values where available.

3. **Fallback to plot-dataset values**

- For any movie where IMDb does not provide genres, actors, or directors, fill out those fields using the corresponding values already present in the movie's plot dataset (if available).

4. **Remove remaining records with missing core metadata**

- After merging + fallback, drop movies still missing genre, actor, or director information. **345** movies were excluded

5. **Format multi-valued fields**

- Store multiple genres, multiple actors, and multiple directors as **comma-separated strings** in their respective columns (e.g., "Drama, Romance", "Actor A, Actor B, Actor C").

6. **Validation**

- All titles are between 1970–2017.
- No duplicate (Title, ReleaseYear) pairs remain.
- No missing values in plot/genres/actors/directors.

Final Features: Two new features were created to be used as predictor variables:

- **released_before_2000:** A binary variable indicating whether a movie was released before the year 2000.
- **combined_text:** A combined text feature that merges the movie plot, genres, actors, and director information.

Supervised Learning

Methods description

Supervised Learning Workflow and Feature Representations:

1. **Splitting the movie plot into individual chunks:** To build the supervised model, each movie plot was split into individual sentences, creating multiple rows per movie. For example, a 20-sentence plot generated 20 records for the same movie.

2. **Creating training and test data:** An 80/20 split was used for training and testing the data. Before splitting, movies with fewer than five rows were removed from the dataset.

The **training data** included the features created during the feature engineering step: combined_text, which merges the movie's plot chunks, genre list, actor list, and director list, and released_before_2000.

The **testing data** required additional transformation to simulate user input at each stage of the five-question response sequence.

- **Test_df1:** Simulates the user's initial raw text input using movie plot chunks (combined_text). The released_before_2000 feature is set to 2 to indicate missing user input. This dataset is used to generate the first prediction and evaluate accuracy and F1 score.
- **Test_df2:** The combined_text from Test_df1 and the released_before_2000 feature (with values of 1 or 0) was used to generate the second prediction and evaluate its accuracy and F1-score.
- **Test_df3:** combined_text from test_df2, appended with one of the randomly chosen genres from the movie's genre list and the released_before_2000 flag (1 or 0), is used to generate the third prediction and calculate its accuracy and F1-score.
- **Test_df4:** combined_text from test_df3, appended with one of the randomly chosen actors from the movie's actors list and the released_before_2000 flag (1 or 0), is used to generate the fourth prediction and calculate its accuracy and F1-score.
- **Test_df5:** combined_text from test_df4, appended with one of the randomly chosen directors from the movie's director list and the released_before_2000 flag (1 or 0), is used to generate the fifth prediction and calculate its accuracy and F1-score.

3. Transformation:

- Vectorize the train and test combined text using TF-IDF vectorization with min_df=5, stop_words='english', max_features=3000
- Some of the models require the target variable in numerical format (i.e., Xgboost, LinearSVC, etc.). Label encoding was performed on the test and training labels.

Post transformation, there are **3001 predictor variables** – 3000 text vector variables and 1 binary 'released_before_2000' variable.

4. Building the prediction models and refining the best model through hyper-tuning

5. Calculating evaluation metrics using 5-fold cross-validation train and test data samples for the best models obtained in step 4.

6. Detailed analysis of the best model

Learning Methods and Why they are chosen for the Movie dataset:

DummyClassifier: Baseline model making random predictions with uniform probability; establishes a performance benchmark.

SGDClassifier: Linear, probabilistic model (logistic regression) optimized for high-dimensional, sparse data; supports parallel computation and large-scale learning.

LinearSVC: Linear, non-probabilistic SVM classifier for sparse data; fast, scalable, supports multi-class classification, and integrates well with preprocessing pipelines—suitable for predicting movie names from plots and metadata.

XGBClassifier: Tree-based gradient boosting model handling high-dimensional, sparse data efficiently; achieves strong predictive performance on text-based features and supports parallel computation.

Hyperparameter Tuning the models:

The model's hyperparameters were optimized using three stratified dataset splits to preserve class proportions. RandomizedSearchCV was employed within the optimal range, and accuracy was used to select the best-performing model. Table 1 summarizes the hyperparameter settings for each model.

Model	Hyperparameters
DummyClassifier	NA
SGDClassifier	"alpha": [0.0001, 0.001, 0.01, 1]
LinearSVC	"C": [0.01, 0.1, 1]
XGBClassifier	"max_depth": [6, 10]

Table 1: Hyperparameters used for model tuning

Supervised Evaluation

Evaluation Metrics:

We used Accuracy and F1-score as evaluation metrics. The following are the justifications for using these evaluation metrics:

1. **Simple and intuitive:** The goal of the model is to predict the movie name based on user inputs. Either it predicts the movie correctly, or it doesn't. Accuracy directly answers that question.
2. **Evaluating class imbalance in the training data:** The dataset contains over 8,000 movies, and splitting plots into sentences generates roughly equal rows per movie, ensuring no class imbalance. Similar accuracy and F1-score values (Table 2) confirm this..

Summary of results:

LinearSVC outperformed all other models across the five test sets. On test set 5, it achieved a mean accuracy and F1-score of 0.82, indicating consistent performance across classes. The best model was obtained with $C = 1$. Details

of the LinearSVC model and its structure are provided in the appendix and illustrated in Image 1.

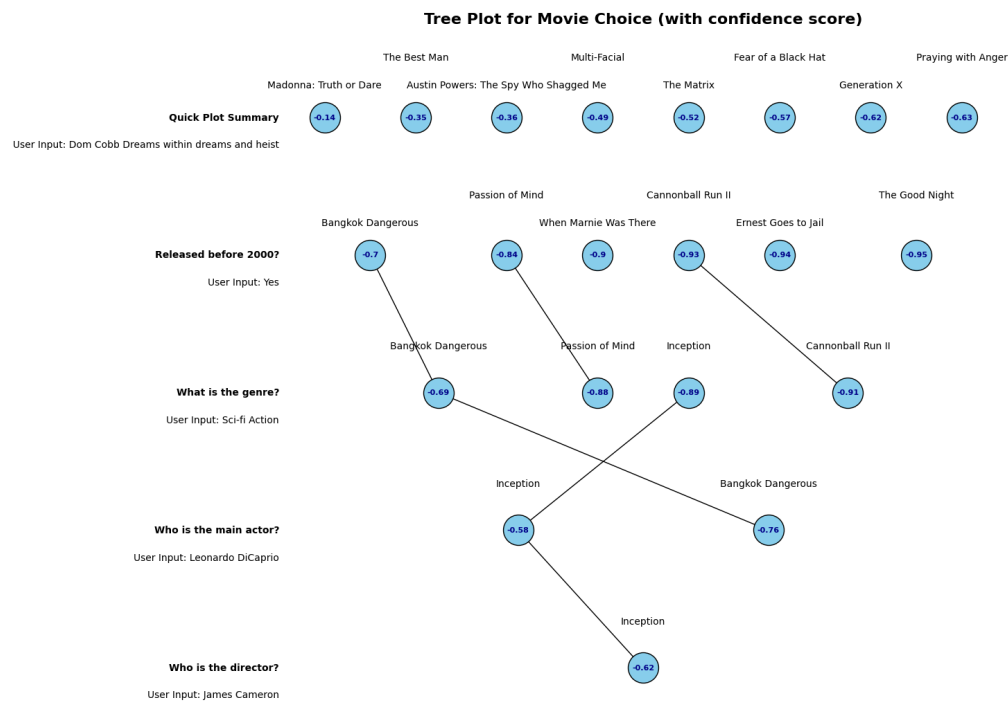


Image 1: tree plot showing the results of the LinerSVC model for a user input

Comparing Evaluation Metrics across the Best Models in the Model Families:

Table 2 compares the mean accuracy, F1-score, and standard deviation of each best model across 5-fold cross-validation samples.

Model	Measurement	Test1	Test2	Test3	Test4	Test5
DummyClassifier	Mean Accuracy	0.00013	0.00013	0.00013	0.00013	0.00013
	Accuracy Std-dev	0	0	0	0	0
	Mean F1-score	0.00013	0.00013	0.00013	0.00013	0.00013
	F1-score Std-dev	0	0	0	0	0
LinearSVC	Mean Accuracy	0.0038	0.0784	0.1185	0.5155	0.8210
	Accuracy Std-dev	0.0011	0.0011	0.0011	0.0011	0.0003

	Mean F1-score	0.0036	0.0774	0.1158	0.5216	0.8200
	F1-score Std-dev	0.0011	0.0007	0.0012	0.0016	0.0004
SGDClassifier	Mean Accuracy	0.0035	0.0207	0.0314	0.1652	0.3060
	Accuracy Std-dev	0.0003	0.0019	0.0029	0.0076	0.0092
	Mean F1-score	0.0007	0.0151	0.0222	0.1445	0.2868
	F1-score Std dev	0.0001	0.0022	0.0032	0.0091	0.0107
XGBClassifier	Mean Accuracy	0.0016	0.0015	0.0053	0.0472	0.1244
	Accuracy Std-dev	0.0003	0.0003	0.0001	0.0014	0.0014
	Mean F1-score	0.0016	0.0014	0.0036	0.0388	0.1077
	F1-score Std dev	0.0002	0.0003	0.0002	0.0012	0.0008

Table 2: Mean Accuracy, F1-score score and their standard deviations comparison across models using 5-fold cross-validation

In-Depth Evaluation

Feature Importance:

Applying feature importance to tf-idf word vectors and binary predictor variable released_before_2000 resulted in the following top 10 features for the LinearSVC model (Table 3).

Feature	Type	Importance
john	Tf-Idf vector	0.1773
released_before_2000	Binary	0.1755
michael	Tf-Idf vector	0.1736
comedy	Tf-Idf vector	0.1668
drama	Tf-Idf vector	0.1612

david	Tf-Idf vector	0.1587
romance	Tf-Idf vector	0.1581
crime	Tf-Idf vector	0.1516
action	Tf-Idf vector	0.1448
thriller	Tf-Idf vector	0.1421

Table 3: Top 10 features of the LinearSVC model

Ablation Analysis:

Ablation analysis was performed on the LinearSVC model. The goal of this analysis was to evaluate feature importance by removing one or more predictor variables at a time and assessing their impact on model performance.

Table 4 presents the results of this analysis.

Approach	Combined_text	Released_before_2000	Accuracy On Test5	F1-Score On test5	Insights
Dropping Released_before_2000	Included in tfidf vectors form	Not included	0.8002	0.8002	A slight drop in accuracy compared to when all were included.
Dropping combined_text	Not included	Included in binary form	0.0025	0.0000	Most important feature
All included	Included in tfidf vectors form	Included in binary form	0.8190	0.8172	

Table 4: Ablation analysis showing that combined_text is the most important feature

Sensitivity Analysis & Tradeoffs: Hyper-tuning the LinearSVC regularization parameter (C) shows that higher C values increase both training time and accuracy. Since the accuracy gain from C = 0.1 to C = 1 is small but training time rises significantly, C = 0.1 is preferred to balance performance and efficiency.

C	Rank	Mean Fit time (Sec)	Mean Score time (Sec)	Mean Test Accuracy
0.01	3	4717.6	12.93	0.9467
0.1	2	3670.8	12.91	0.9943
1	1	3899.9	13.0	0.9989

Table 5: Sensitivity analysis was obtained through the cv_results_ attribute of RandomizedSearchCV. It captures model performance across different hyperparameter combinations.

Failure analysis

1. The title ‘Stone’ has appeared 83 times in the test_df5 dataset. The following are the predictions for these 83 records in Table 6:

Prediction for ‘Stone’	Count	Avg confidence score
Stone	56	0.4569
The Painted Veil	26	0.4577
Snowden	1	0.2153

Table 6: Predictions for the movie ‘Stone’.

It’s an example of the **Systematic Error** category. “Stone” is repeatedly confused with “The Painted Veil” — not random noise. This suggests that the model learned similar feature patterns between those two titles. This is true because both “Stone” and “The Painted Veil” were released after the year 2000 (released_before_2000 = 0), have the same actor, Edward Norton, and the same director, John Curran.

An analysis of top text features that pushes the model to misclassify “Stone” as “The Painted Veil” confirms the same in Table 7:

Feature	Avg Contribution
tfidf__curran	0.9360
tfidf__norton	0.3466
tfidf__edward	0.2125
tfidf__john	0.1572

Table 7: Features responsible for the incorrect prediction of “Stone” as “The Painted Veil”

How to prevent: Systematic confusion arises when the model learns overlapping features between classes. Adding more discriminative features, increasing training examples, or using embeddings to better represent the plot content could help prevent this type of error.

2. The title ‘Movie 43’ has appeared 21 times in the test_df5 dataset, of which 20 resulted in incorrect predictions across 16 different movies. The top five prediction records are shown in **Table 8**:

Top 5 Prediction for ‘Movie 43’	Count	Avg Confidence Score
Our Son, the Matchmaker	1	0.2349
The Reluctant Fundamentalist	1	0.2247
What a Girl Wants	2	0.1819

Terminator Salvation	1	0.1768
Going the Distance	1	0.1710

Table 8: Top 5 Predictions for the movie ‘Movie 43’. All of them has low confidence score.

It’s an example of **the Random Error** failure category. The misclassifications are spread across many Titles, and each prediction has a low confidence score (avg score = 0.1448). This indicates overlapping features and ambiguous plot summaries across these movies.

How to prevent: A splitting strategy with multiple sentences could have generated a more contextual and meaningful training and test dataset to resolve the issue.

3. The title ‘Living Out Loud’ has appeared only once in the test_df5 dataset. It shows a wrong prediction of ‘Birth of The Beatles’ with a confidence score of 0.0174. Further inspection of training data (train_df) shows that there were only 5 training data points of the ‘Living Out Loud’. It’s an example of the **Edge Case** failure category, where the model cannot generalize due to **scarce training data**.

How to prevent: Edge-case failures happen when the model has too few training examples to generalize. Increasing data for rare classes, using data augmentation, or leveraging pre-trained models can help reduce these errors

Unsupervised Learning

Unsupervised Learning Workflow and Feature Representations:

The unsupervised learning workflow is designed to be able to recommend a list of five movies should the supervised learning models fail to guess the correct movie from user inputs. The unsupervised learning workflow contains three main steps: feature representations, clustering, and cosine similarity for final output.

Feature Representations

The feature representations for the clustering and cosine similarity steps in the unsupervised learning workflow were created by first creating individual vector representations of the ‘Release Year’, ‘Plot’, ‘Cast’, and ‘Genre’ fields. Through trial and error, we found that transforming a single string into a vector led to poor clusters and found the best clusters were created when each field is vectorized individually before being combined for the final vector.

The clustering step uses two fields: ‘Plot’ and ‘Genre’. In order to determine how to create the best performing feature representation for the clustering step, we tried three different text embedding models for the ‘Plot’ field ('fasttext-wiki-news-subwords-300', 'word2vec-google-news-300', ‘Text8’ Word2Vec) and two different vectorizers for the ‘Genre’ field (MultiLabelBinarizer model, TfidfVectorizer). We then tested the six combinations using a KMeans(n_clusters = 25) model and evaluated the model using Silhouette, Davies-Bouldin, and Calinski-Harabasz scores to determine which model performed the best. Based on our results, we decided to use the FastText model for

the ‘Plot’ fields and TfidfVectorizer for the ‘Genre’ field. After combining the ‘Plot’ and ‘Genre’ vectors into X_cluster, we noticed X_cluster had a large number of dimensions and decided to apply Truncated SVD for better clustering algorithm performance. In the same method we used to evaluate vectorizers, we explored a range of n_components from 5 to 100 and found that n_components = 5 gave us the best results. The final feature representation for our clustering step was then labelled X_cluster_reduced.

The cosine similarity step uses four fields: ‘Plot’, ‘Genre’, ‘Release Year’, and ‘Cast.’ Similar to how we calculated X_cluster, we decided to transform each field into a vector separately before combining the results into one vector. For the ‘Plot’ field, we decided to use the same text embedding model we found produced the best results, previously: FastText. We decided to continue using a TfidfVectorizer for the ‘Genre’ field and decided to use another TfidfVectorizer for the ‘Cast’ field and transform ‘Release Year’ with a StandardScaler. We then decided to apply Truncated SVD to the combined X_cossim vector to create X_cossim_reduced.

However, since the cosine similarity calculations needed to take place within each cluster, we calculated an X_cossim_reduced for each cluster and trained individual field transformers and truncated SVD for each cluster. The ‘Plot’ field, however, used the same FastText embedding model for all clusters.. We then stored the X_cossim_reduced values in a dictionary and stored the different field transformers and truncated SVD in dictionaries for later access.

Clustering

For the initial exploration of the clustering step of the unsupervised learning workflow, we assessed four different clustering algorithms: K-Means, DBSCAN, Agglomerative, and Gaussian Mixture. In order to find the best model for the K-Means, Agglomerative, and Gaussian Mixture Model, we tested a range of k-values from 5 to 250 in increments of 5 to find the model that had the best combination of high Silhouette score, low Davies-Bouldin score, and high Calinski-Harasz score. In order to find the best DBSCAN model, we tested every combination of epsilon values between 0.1 and 1 and minimum samples between 1 and 40 in increments of 5 to, again, find the model that had the best combination of scores.

Model(s)	Hypertuning Parameters
KMeans, Agglomerative, GMM	“n_clusters/n_components” : [5, 250, interval = 5]
DBSCAN	“eps” : [0.1, 1, interval = 0.1] “min_samples” : [1, 40, interval = 5]

Cosine Similarity

During the feature representation step of the workflow, we calculated X_cossim_reduced for each cluster and stored the matrices in a dictionary for later access. We also stored a cast vectorizer, year scaler, and truncated SVD model for each cluster in three additional dictionaries.

For the cosine similarity step of the workflow, we then created a function to return the five most similar movies to the user’s inputs. The first step of this function takes a user’s inputs and a model input to predict which cluster the

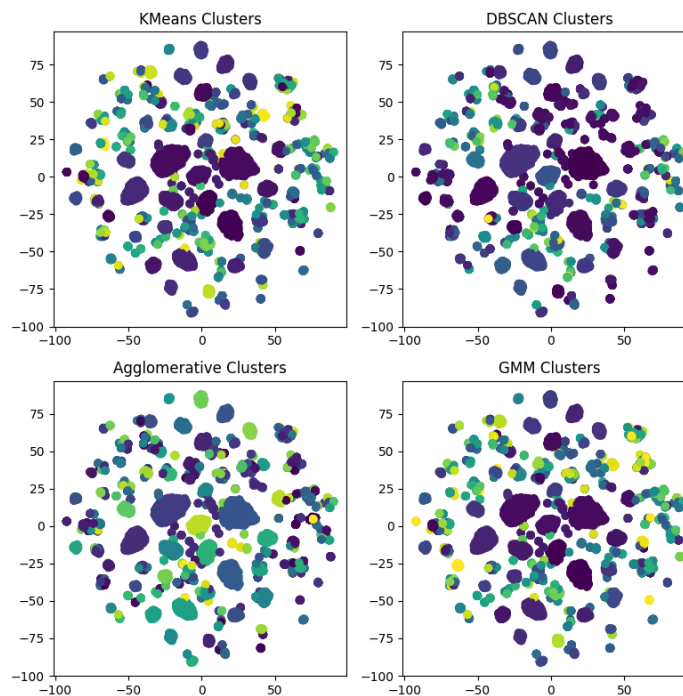
movie the user is describing is in. Once the cluster is determined, the function creates a vector using the user's inputs and the predicted cluster's cast vectorizer, year scaler, and truncated SVD from the previously created dictionaries. The function then calls `X_cossim_reduced` for the predicted cluster. The function then runs a cosine similarity calculation and returns the five movies with the highest similarity scores.

Using this function, we are then able to take a user's inputs along with our best model from the clustering step of the workflow to return five movies similar to the movie the user is describing with hopefully, one movie in the list being a match for the user's movie.

Unsupervised Evaluation

In order to evaluate the performance of our clustering algorithms, we decided to calculate the Silhouette score, Calinski-Harabasz score, and Davies-Bouldin score for each algorithm. The average Silhouette score of each model will allow us to evaluate the similarity of points within a cluster compared to points outside of the cluster with a higher Silhouette score value indicating well-defined clusters. The average Davies-Bouldin score of each model will allow us to evaluate how separate the model's clusters are from each other with lower values indicating well-separated clusters. The average Calinski-Harabasz score for each model will allow us to evaluate the ratio of the average distance of points within a cluster to the inter-cluster dispersion between all clusters with higher numbers indicating well-separated, dense clusters. Each metric will give us a clearer picture of the clustering algorithm's overall performance.

After hypertuning each of the four different clustering algorithms and finding the best model, we recorded the scores for comparison and visually compared the scatterplot outputs of each clustering algorithm.



After visually evaluating the scatterplots created by each of the clustering algorithms, we discover that each of the algorithms seemed to perform similarly to the others. The scatterplot visualizations of the KMeans algorithm and GMM algorithm seem to be almost indistinguishable from each other. After calculating the Silhouette, Davies-Bouldin, and Calinski-Harabasz scores, we found that the scores supported our observations.

Best Model	Silhouette Score	Davies-Bouldin Score	Calinski-Harabasz Score
KMeans (n_clusters = 155)	0.816	0.444	21,124.362
DBSCAN (eps = 0.1, min_samples = 1)	0.733	0.478	2,660.982
Agglomerative (n_clusters = 145)	0.823	0.454	19,887.270
Gaussian Mixture (n_clusters = 160)	0.811	0.444	21,049.886

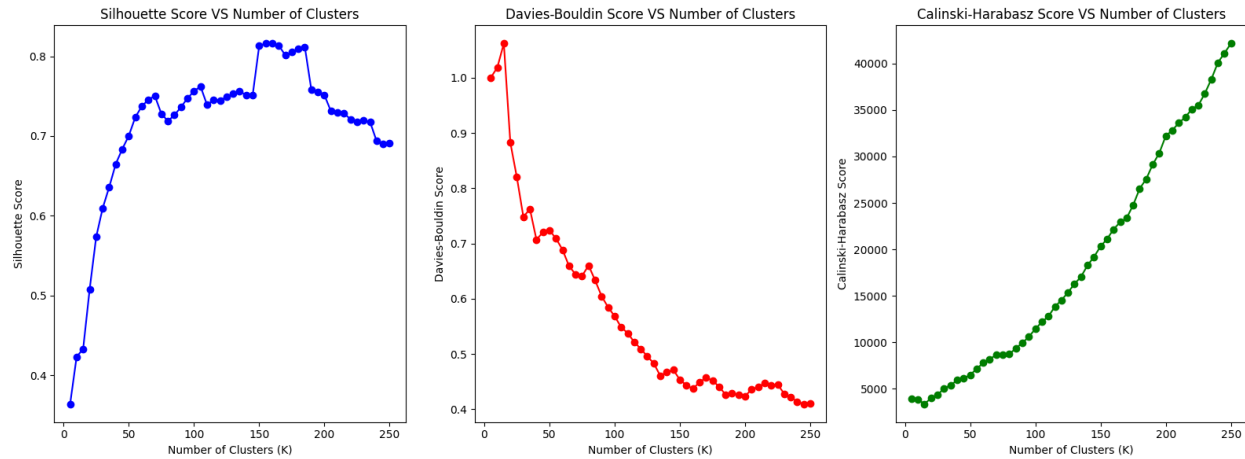
The Davies-Bouldin scores for all four algorithms were very similar and with a range of 0.44-0.48 with the KMeans Algorithm and GMM algorithm sharing the same score. There was a little more variation in the Silhouette scores with the KMeans algorithm, Agglomerative algorithm, and GMM algorithm scoring within a range of 0.81 and 0.83, while the DBSCAN algorithm performed comparatively worse with a score almost 0.1 below the range of the other algorithms. The Calinski-Harabasz scores had the largest range with DBSCAN performing poorly compared to the other models and KMeans performing the best, just around 80 points over the GMM algorithm's score.

Overall, we determined that all the clustering algorithms performed reasonably well but that the KMeans model performed the best with a high Silhouette score, the lowest Davies-Bouldin score, and the highest Calinski-Harabasz score and should be used in the cosine similarity step of the workflow.

Sensitivity Analysis

After completing a sensitivity analysis on the best clustering algorithm: KMeans(n_cluster = 155), we found that the model is highly reliant on the 'Genre' field. After training the model on a dataset that only includes 'Plot' values, the calculated silhouette score was -0.038 indicating that without 'Genre' data, the clustering algorithm is unable to create clear, well-defined clusters. When doing the reverse and training the model on a dataset that only includes 'Genre' values, the calculated silhouette score was 0.603 which is lower than the model's original silhouette score of 0.816 but still indicates the clustering algorithm is able to create clusters reasonably well. This indicates that the model is very sensitive to changes in the 'Genre' field.

Additionally, the model seems to be sensitive to changes in the number of clusters. A higher number of clusters leads to a lower Davies-Bouldin score and higher Calinski-Harabasz score and there is a range where Silhouette score is maximized.



Discussion: Supervised Learning

Surprising elements about the results:

- **LinearSVC** significantly outperformed all other models.
- **XGBClassifier** showed the worst performance despite having the longest training time and the largest model size.

Challenges and Responses:

1. Long Compute Time During Model Training and Hyperparameter Tuning

Response:

- Reduced the dataset to include only movies released after 1969.
- Limited the TF-IDF feature space to 3,000 vectors.
- Parallelized training jobs across multiple CPU cores.
- Restricted the number of hyperparameter combinations to reduce computation time.
- Executed hyperparameter tuning separately for each model instead of combining them in a single script.
- For XGBClassifier tuning, a high-memory instance (1,500 GB RAM, multiple cores) was used, and the job was scheduled to run for 15 hours.

2. Running 5-Fold Cross-Validation to Compute Mean Accuracy, F1-Score, and Standard Deviation

Response:

- Executed each fold in parallel using the Parallel function from joblib.
- Used a large-memory instance (1,500 GB RAM, multiple cores) and scheduled the job to run for 10 hours.

Extending the solution with more time and resources

- Experiment with additional vector embedding techniques (e.g., Word Embeddings – Word2Vec, Contextual Embeddings – BERT, Sentence Embeddings – Sentence-BERT) to improve accuracy.
- Incorporate movie quotes and dialogues alongside plot summaries for broader contextual understanding
- Perform hyperparameter tuning with a wider search space.
- Explore alternative models, such as Neural Networks.

- Expanding Ablation analysis by using tf-idf vectors feature
- Develop a user interface that allows users to input data and view the model's predictions.

Discussion: Unsupervised Learning

Surprising elements about the results:

One of the most surprising elements about the results were that all four clustering algorithms (KMeans, DBSCAN, Agglomerative, and Gaussian Mixture) performed similarly to each other and had similar Silhouette, Davies-Bouldin, and Calinski-Harabasz score results. We thought that there would be a clear best clustering algorithm but found that KMeans performed only slightly better from our interpretation of the scores.

Overall, our results seemed to indicate that the structure of the input vector (X_cluster), configuration of Truncated SVD, and number of clusters had more of an impact on the model performance than the algorithm.

Challenges and Responses:

One of the biggest challenges creating the unsupervised learning workflow was finding an approach that worked well for our data. Our original approach was to create one large string value from the fields in our data and use a single vectorizer to create an input to train our clustering algorithms on. When we attempted this method, we found the performances of our models to be very poor and we were unable to predict five movies that we considered similar to our test move 'Inception.' When inputting our test case into the model, we would get results within the romantic comedy genre which is very dissimilar to the type of movie that 'Inception' is.

In order to solve this problem, we decided to transform each field separately before creating our final input. This gave us more control on how the model input was structured and allowed us to use a vectorizer/word embedding model that would work best for each field.

This line of thinking led us to incorporating a cosine similarity step into our unsupervised learning workflow. By breaking down the problem into two steps, we were able to design each step for a specific purpose (clustering similar movies, identifying the movie within the most likely cluster). This allowed us to get much better results and even led to our model being able to successfully predict 'Inception' if given enough data.

Extending the solution with more time and resources

- Incorporate more movie data for clustering algorithm training to allow the algorithm to learn more cluster patterns
- Perform additional feature exploration and model hypertuning to continue to improve the unsupervised workflow's performance
- Explore additional model types, word embedding models, and vectorizers to create inputs that allow the clustering algorithms to perform better
- The current clustering algorithm is highly reliant on the 'Genre' field, however it would be better if the model relied on the 'Plot' field more when clustering so the user does not have to input more than one genre for the model to find their movie.

Ethical Considerations

The main ethical concern stems from **limiting the dataset to American movies released after 1969**, which excludes older and international films. This restriction could be perceived as a bias toward American movies and limit the enjoyment or relevance for users who prefer other films. The supervised model's predictions and the unsupervised recommendation system are both affected by this limitation.

The group emphasizes that this choice was due to **data, resource, and time constraints**, not value judgment. Including a **disclaimer** explaining the dataset scope and the reasons for these limitations can mitigate misunderstandings. Expanding the dataset in the future would allow for more inclusive and accurate predictions..

Statement of Work

Rashid Zaheer	Freya Patel	Andrew Rebits
- Initial project idea and group creation - Completed initial data preparation and cleaning - Created and fine-tuned supervised learning model - Report sections: Feature Engineering, Supervised Learning, Supervised Learning Discussion	- Completed unsupervised learning feature engineering - Created and fine-tuned unsupervised learning model(s) - Report sections: Unsupervised Learning, Unsupervised Learning Discussion	- Data preparation for report visualizations - Created report visualizations - Report Sections: Introduction, Related Work, Data Sources, Ethical Considerations
All: Created initial project proposal, Attended regular project meetings, Participated in Standup 1 and 2		

Appendix

LinearSVC Model Summary

Pipeline Steps:

- *preprocess: ColumnTransformer*
- *model: LinearSVC*

Model Parameters:

C: 1
class_weight: None
dual: auto
fit_intercept: True
intercept_scaling: 1
loss: squared_hinge
max_iter: 1000
multi_class: ovr
penalty: l2
random_state: None
tol: 0.0001
verbose: 0

Learned Coefficients Shape: (7590, 3001)

Preprocessing Transformers:

*ColumnTransformer(transformers=[('tfidf',
TfidfVectorizer(max_features=3000, min_df=5,
stop_words='english'),
'combined_text'),
('year', 'passthrough',
['released_before_2000'])])*

References

Akinator: The Guessing Game. Akinator, 2023, <https://en.akinator.com/>.

Filmillion: Movie Recommendation Engine. Filmillion, 2023, <https://filmillion.com/>.

Gensim Word2vec embeddings (n.d.). Radimrehurek.com. <https://radimrehurek.com/gensim/models/word2vec.html>

IMDb Datasets. IMDb, 2023, <https://datasets.imdbws.com/>.

Movie Recommender Template. Easy-Peasy, 2023, <https://easy-peasy.ai/templates/movie-recommender>.

Priya, Vishnu. "Wiki Movie Plots with Summaries." Hugging Face, 2023, <https://huggingface.co/datasets/vishnupriyavr/wiki-movie-plots-with-summaries>.

T, Jessie. "Famous Movie Quotes." Hugging Face, 2023, https://huggingface.co/datasets/jtatman/famous_movie_quotes.

Tree Plots in Python. Plotly, 2023, <https://plotly.com/python/tree-plots/>.

Waskom, Michael. "seaborn.heatmap." Seaborn Documentation, 2023, <https://seaborn.pydata.org/generated/seaborn.heatmap.html>.

Waskom, Michael. "seaborn.scatterplot." Seaborn Documentation, 2023, <https://seaborn.pydata.org/generated/seaborn.scatterplot.html>.